

前端开发规范 (React + TypeScript)

一、技术栈约定

- 框架：React 18 + TypeScript
- 构建：Vite
- 路由：React Router
- UI 组件库：统一使用团队选定的组件库，禁止随意混用多套 UI 库
- 状态管理：优先使用组件本地状态与 Context，复杂场景再引入状态库

二、目录结构

- ``api/``：接口封装与请求实例
- ``components/``：可复用的通用组件
- ``pages/``：页面级组件，按业务模块分子目录
- ``layouts/``：布局组件
- ``utils/``：工具函数
- ``types.ts``：全局类型定义

三、命名与代码风格

- 1 组件文件与组件名使用大驼峰，如 ``MessageBubble.tsx``。
- 2 工具函数、变量使用小驼峰，如 ``formatDate``。
- 3 常量使用全大写下划线。
- 4 类型/接口使用大驼峰，接口不强制加 `I` 前缀。
- 5 使用函数式组件与 Hooks，避免 `class` 组件。

四、TypeScript 要求

- 开启严格模式 (strict)，杜绝隐式 any。
- 公共函数、组件 Props 必须有明确类型。
- 优先使用 interface 描述对象结构，联合类型使用 type。
- 避免使用 ``as any`` 绕过类型检查，确需断言时注明原因。

五、组件设计原则

- 单一职责：一个组件只做一件事，过大时拆分。
- 展示与逻辑分离：数据请求与业务逻辑尽量抽离到 hooks 或 api 层。
- Props 只读，状态变更通过回调上抛。
- 列表渲染必须提供稳定的 key，避免用索引作为 key。

六、样式与可读性

- 遵循团队既定的配色主题，保证核心组件的可读性与对比度。
- 避免过度透明或低对比度导致按钮"隐形"、文字模糊。
- 移动端与桌面端均需验证布局，避免溢出与错位。

七、性能优化

- 合理使用 memo / useMemo / useCallback，避免不必要的重渲染。
- 大列表使用虚拟滚动。
- 图片与资源按需加载，路由级代码分割。

八、提交要求

- 通过 ESLint 与 TypeScript 类型检查
- 无 console.log 调试残留
- 交互与边界状态（加载中、空数据、错误）均已处理